Business Yard Calculator – High-Level Design (HLD)

Status: v1.0 (decisions confirmed) · Stack: Vue.js + Tailwind CSS only (no backend)

1. Overview

A single-page phone calculator for pricing fabric by the yard. The user types yards with + (for example `2 + 3`), then presses **Business Amount** instead of equals. The app adds the yards, picks the rate, and shows total yards and total amount in ks; at 10 yards or more it also shows a free amount. The UI is phone-only, in Burmese and English.

2. Goals and Non-Goals

Goals

- Fast, mobile-friendly calculator UI.
- Correct decimal handling (no floating-point surprises).
- Configurable business rate (default 4000 ks per yard).
- Automatic discount rule for large quantities.

Non-Goals (v1)

- No backend, login, or database.
- No multi-user sync.
- No calculation history.
- No desktop layout.

3. Business Rules

Rule	Description
BR-1	Yards are typed as yards.inches: the digits after the dot are inches (5.23 = 5 yards 23 inches).
BR-2	1 yard = 36 inches. When the inches total 36 or more, each 36 inches becomes 1 yard (4 + 5.23 + 3.24 = 13 yards 11 inches).

Rule	Description
BR-3	Only + in v1. The equals button is replaced by a button labelled with the current rate (default 4000).
BR-4	Total amount = total inches × base rate ÷ 36 (exact), then rounded up to the next 50 ks (53,222 → 53,250). Base rate defaults to 4000 ks per yard, editable in Settings.
BR-5	At or above the threshold (default 10 yards = 360 inches), Total free amount = total inches × discount rate ÷ 36 (exact), rounded up to the next 50 ks. Discount rate defaults to 3900 ks, editable in Settings.
BR-6	The threshold is checked on the total of the whole equation (6 + 4 = 10).
BR-7	Off = exact amount at the base rate − exact amount at the discount rate (before rounding), then rounded up to the next 50 ks.
BR-8	Below the threshold the result shows Total yards and Total amount. At or above it, the result also shows Total free amount and Off.

Worked examples

- `4 + 5.23 + 3.24` → 13 yards 11 inches (479 inches); Total amount 53,250 ks (exact 53,222.22); Total free amount 51,900 ks (exact 51,891.67); Off 1,350 ks (exact 1,330.56)
- `2 + 3` → 5 yards; Total amount 20,000 ks
- `1.33 + 1.4` → 3 yards 1 inch (109 inches); Total amount 12,150 ks (exact 12,111.11)
- `6 + 4` → 10 yards; Total amount 40,000 ks; Total free amount 39,000 ks; Off 1,000 ks
- `12` → 12 yards; Total amount 48,000 ks; Total free amount 46,800 ks; Off 1,200 ks

4. User Flow

1. User enters yards with the keypad and the + button.
2. The display shows the expression live.

3. User presses the rate button (default 4000).
4. The app sums the yards in inches and computes the amounts.
5. The result card shows Total yards and Total amount; at 10 yards or more it also shows Total free amount and Off.
6. User clears and starts again. No history is kept.

5. Architecture

A client-side only Vue 3 application.

- **View layer:** Vue components styled with Tailwind.
- **State layer:** a small reactive store (Vue `ref` / `reactive` or Pinia) holding expression, rate, discount settings, and result.
- **Logic layer:** pure functions (expression parser and pricing calculator) kept separate from components so they are easy to test.
- **Persistence:** `localStorage` for the configured rate and discount settings.

6. Components

Component	Responsibility
`CalculatorPage`	Page layout and wiring.
`DisplayPanel`	Shows the live expression.
`Keypad`	Digits, decimal point, +, clear, backspace.
`BusinessAmountButton`	Triggers the calculation (replaces equals).
`ResultCard`	Total yards, total amount, and free amount when the discount applies.
`SettingsPage`	Edit base rate, discount rate, threshold, and language.

7. Calculation Design

- **Parsing:** a tokenizer reads each number as yards.inches and converts it to inches (`yards × 36 + inches`). No `eval` .

- **Exact math:** all sums are whole inches, so no floating-point errors. Amounts stay exact (as fractions of 36) until the final rounding.
- **Total amount:** `ceilTo50(totalInches × baseRate ÷ 36)`.
- **Discount:** if `totalInches >= threshold × 36`, then `freeAmount = ceilTo50(totalInches × discountRate ÷ 36)` and `off = ceilTo50(totalInches × (baseRate − discountRate) ÷ 36)`.
- **Rounding:** always up to the next 50 ks; the step is fixed, not a setting.
- **Display:** `floor(totalInches ÷ 36)` yards and `totalInches mod 36` inches, for example 479 inches shows as 13 yards 11 inches.

8. Configuration Model

- `baseRate` (default 4000)
- `discountThreshold` (default 10)
- `discountRate` (default 3900)
- `language` (Burmese or English)

All are editable in Settings and saved in `localStorage`.

9. Validation and Edge Cases

- Empty or incomplete expression (for example `2 +`).
- Multiple decimal points in one number.
- Division by zero (if division is supported).
- Negative or zero yards.
- Very large numbers and display formatting (thousands separators).

10. UI / UX Notes

- Phone-only, portrait layout with large touch-friendly buttons.
- Burmese and English labels from one translation file, switched in Settings.
- Amounts with thousands separators and the "ks" unit.
- Result labels: Total yards, Total amount, and, at 10 yards or more, Total free amount and Off.

11. Testing Strategy

- Unit tests for the parser and pricing logic (including the 10-yard boundary and decimals).
- Component tests for keypad input and the Business Amount button.

12. Delivery Phases

1. Core calculator and pricing logic.
2. Discount rule and decimal handling.
3. Settings panel and persistence.
4. Polish, responsive design, and testing.

13. Decisions and Open Questions

Decided: yards.inches input with 36 inches per yard and inches carrying into yards; at or above the threshold the result adds Total free amount (discount rate) and Off; Off is taken from the exact amounts; all amounts are rounded up to the next 50 ks; threshold checked on the whole-equation total; only + in v1, with a rate-labelled button instead of equals; rates and threshold editable in Settings; no history; Burmese and English; phone only.

Open questions: none. **Next step:**

1. Build plan: project structure, calculation module with unit tests (the examples in section 3 are the test cases), then keypad, result card, and Settings.